

Guida alle librerie C++ in CV+

Librerie statiche, DLL, file header e pacchetti ZIP

1. Dove si trovano le funzioni

Apri la scheda **Estensioni C++**. Da questa finestra puoi importare un file header **.h/.hpp**, una libreria statica **.a**, una libreria dinamica **.dll**, una guida PDF oppure un pacchetto ZIP completo. Quando il pacchetto contiene i file corretti, CV+ li installa e li collega automaticamente alla compilazione.

2. Differenza tra libreria statica e DLL

Libreria statica (.a)	Libreria dinamica (.dll)
Il codice della libreria viene incorporato nel file .exe durante la compilazione.	Il file .exe contiene solo il collegamento; la DLL deve essere presente quando il programma viene eseguito.
Il programma finale è più grande, ma normalmente non deve portare con sé la libreria separata.	Più programmi possono usare la stessa DLL. Il programma finale è più piccolo, ma dipende dal file .dll.
Per MinGW il file tipico è libnome.a .	Con MinGW si usano di solito nome.dll e la libreria di importazione libnome.dll.a .

In pratica: usa una libreria statica quando vuoi distribuire un solo eseguibile. Usa una DLL quando vuoi aggiornare o condividere la libreria separatamente, oppure quando contiene una finestra grafica o funzioni riutilizzate da più programmi.

3. Esempio semplice senza namespace: libreria statica

Creiamo una libreria che espone la funzione **somma**. Non viene usato alcun namespace.

File **calcoli.h**:

```
#ifndef CALCOLI_H
#define CALCOLI_H

int somma(int a, int b);

#endif
```

File **calcoli.cpp**:

```
#include "calcoli.h"

int somma(int a, int b)
{
    return a + b;
}
```

Compilazione della libreria statica in CMD, senza imporre **-std=c++17**:

```
g++ -c calcoli.cpp -o calcoli.o
ar rcs libcalcoli.a calcoli.o
```

File **main.cpp** che usa la libreria:

```
#include <iostream>
#include "calcoli.h"

using namespace std;

int main()
{
    cout << somma(4, 7) << endl;
    return 0;
}
```

Compilazione manuale del programma:

```
g++ main.cpp -L. -lcalcoli -o programma.exe
```

4. Esempio DLL MinGW

Per una DLL servono il file header, il file sorgente, la DLL e la libreria di importazione. Il simbolo di esportazione viene definito nel file header.

File **saluti.h**:

```
#ifndef SALUTI_H
#define SALUTI_H

#ifdef SALUTI_EXPORTS
#define SALUTI_API __declspec(dllexport)
#else
#define SALUTI_API __declspec(dllimport)
#endif

extern "C" SALUTI_API int somma(int a, int b);

#endif
```

File **saluti.cpp**:

```
#define SALUTI_EXPORTS
#include "saluti.h"

extern "C" int somma(int a, int b)
{
    return a + b;
}
```

Compilazione della DLL e della libreria di importazione, senza **-std=c++17**:

```
g++ -shared saluti.cpp -o saluti.dll ^
-Wl,--out-implib,libsaluti.dll.a
```

Per usare la DLL nel programma:

```
#include <iostream>
#include "saluti.h"

using namespace std;

int main()
{
    cout << somma(10, 5) << endl;
    return 0;
}
```

Compilazione manuale:

```
g++ main.cpp -L. -lsaluti -o programma.exe
```

5. Come preparare uno ZIP per CV+

Lo ZIP può contenere direttamente i file oppure organizzarli in sottocartelle. Per una DLL MinGW è consigliato includere:

```
saluti.zip
  saluti.h
  saluti.dll
  libsaluti.dll.a
  Guida_saluti.pdf      (facoltativa)
  manifest.json         (consigliato, ma non obbligatorio per ZIP semplici)
```

Per una libreria statica:

calcoli.zip
calcoli.h
libcalcoli.a
Guida_calcoli.pdf (facoltativa)

6. Importazione in CV+

1. Apri **Estensioni C++**.
2. Premi **Installa pacchetto / ZIP** oppure il comando di importazione adatto.
3. Seleziona lo ZIP senza estrarlo.
4. Controlla che l'estensione risulti installata.
5. Nel main.cpp includi il file header con **#include "nome.h"**.
6. Compila ed esegui. Se la libreria è dinamica, CV+ deve copiare la DLL accanto all'eseguibile temporaneo.

7. Errori frequenti

Il pacchetto non contiene manifest.json: gli ZIP semplici possono comunque essere riconosciuti quando contengono file .h/.hpp, .a, .dll e PDF validi.

Undefined reference: manca la libreria .a o .dll.a, oppure il nome usato con -l non corrisponde.

DLL non trovata: il file .dll non è accanto all'eseguibile.

Punto di ingresso non trovato: la DLL è stata compilata con dipendenze o dichiarazioni non compatibili. Ricompila con la stessa architettura e la stessa toolchain MinGW del programma.